KERV: Kinematic-Rectified Speculative Decoding for Embodied VLA Models

Zihao Zheng[1], Zhihao Mao[2], Maoliang Li[1], Jiayu Chen[1], Xinhao Sun[1], Zhaobo Zhang[1], Donggang Cao[1], Hong Mei[1], †Xiang Chen[1]
[1] School of Computer Science, Peking University, Beijing, China
[2] School of Computer Science, China University of Geosciences (Wuhan), Wuhan, China

Abstract

Vision-Language-Action (VLA) models build a token-domain robot control paradigm, yet suffer from low speed. Speculative Decoding (SD) is an optimization strategy that can boost inference speed. Two key issues emerge when integrating VLA and SD: first, SD relies on re-inference to address token errors, which is computationally expensive; second, to mitigate token errors, the acceptance threshold in SD requires careful adjustment. Existing works fail to address the above two issues effectively. Meanwhile, as the bridge between AI and the physical world, existing embodied intelligence has overlooked the application of robotic kinematics. To address these issues, we innovatively combine token-domain VLA models with kinematic-domain prediction for SD, proposing a kinematic-rectified SD framework named *KERV*. We employ a kinematics-based Kalman Filter to predict actions and compensate for SD errors, avoiding costly re-inference. Moreover, we design a kinematics-based adjustment strategy to dynamically rectify the acceptance threshold, addressing the difficulty of threshold determination. Experimental results across diverse tasks and environments demonstrate that *KERV* achieves 27%∼37% acceleration with nearly no Success Rate loss.

ACM Reference Format:
Zihao Zheng[1], Zhihao Mao[2], Maoliang Li[1], Jiayu Chen[1], Xinhao Sun[1], Zhaobo Zhang[1], Donggang Cao[1], Hong Mei[1], †Xiang Chen[1]. 2026. *KERV*: Kinematic-Rectified Speculative Decoding for Embodied VLA Models. In *63rd ACM/IEEE Design Automation Conference (DAC '26), July 26–29, 2026, Long Beach, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3770743.3804238

1 Introduction

Vision-Language-Action (VLA) models have emerged as the mainstream for embodied intelligence [9, 37]. They propose a **token-domain paradigm** for robot control that unifies visual, textual, and action information as tokens, yet suffers from low inference speed [7, 17, 31]. Existing work accelerates VLA model inference via three key avenues: model architecture innovation [3, 16, 19, 26]; model compression (pruning [24, 29], caching [22, 28], quantization [11, 18]); and runtime optimizations (layer-skipping [34], early-exit [21, 30] strategies).

Speculative Decoding (SD) is a promising decoding optimization technique that can effectively accelerate Large Language Model (LLM) inference speed. Specifically, it uses a draft model to rapidly generate tokens and then efficiently verifies in parallel by LLMs [10, 32]. However, adapting SD to VLA models is non-trivial. First, SD relies on re-inference to address the token errors during VLA's action generation, which leads to a high computational cost and limits the speed. Second, to mitigate token errors, the acceptance threshold in SD requires careful adjustment to achieve an optimal trade-off between inference speed and task accuracy. Existing works such as Spec-VLA [25] adopt relaxed acceptance to mitigate token errors, yet they set a static acceptance threshold, ignoring the complex variations in tasks and environments. Moreover, Spec-VLA also needs re-inference when facing token errors, which limits its performance.

Meanwhile, although embodied intelligence is widely recognized as the bridge linking AI to the physical world, most existing solutions fail to acknowledge the fundamental importance of kinematics. Traditional robotic control [1, 8] adopts a **kinematic-domain paradigm**, utilizing kinematic information (e.g., joint angles, link displacements, motion velocities) to directly regulate robot movements via pre-defined mathematical models and control laws. It relies on accurate robot dynamic modeling and real-time sensor feedback to achieve precise trajectory tracking and motion execution. Unlike VLA models, it cannot conduct long-term task planning or intelligent environmental perception, but enables accurate motion control in shorter action contexts with lower computational complexity and high efficiency. **This inspires us to combine the token-domain VLA paradigm with the kinematic-domain traditional control paradigm, leveraging the strengths of both.**

In this paper, we take SD optimization as the starting point, innovatively combine the VLA model with a kinematic-based method, and propose a kinematic-rectified SD framework termed *KERV*. (1) First, we analyze VLA model generation errors and acceptance thresholds from both token and kinematic perspectives, finding kinematic-based methods suitable as an adjustment and compensation strategy during VLA generation. (2) Second, we propose a kinematic-based compensation mechanism for token errors in SD instead of relying on re-inference, thereby addressing the challenge of token errors and enabling acceleration. (3) Third, we propose a kinematic-based strategy to dynamically rectify the acceptance threshold during inference, thereby solving the challenge of determining this threshold while maintaining a high success rate across multiple tasks. Moreover, we design a dedicated hardware mapping for the *KERV* framework based on its computational characteristics and implement it on existing hardware platforms. We evaluate

† Corresponding author: Xiang Chen, <xiang.chen@pku.edu.cn>

DAC '26, Long Beach, CA, USA

ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804238

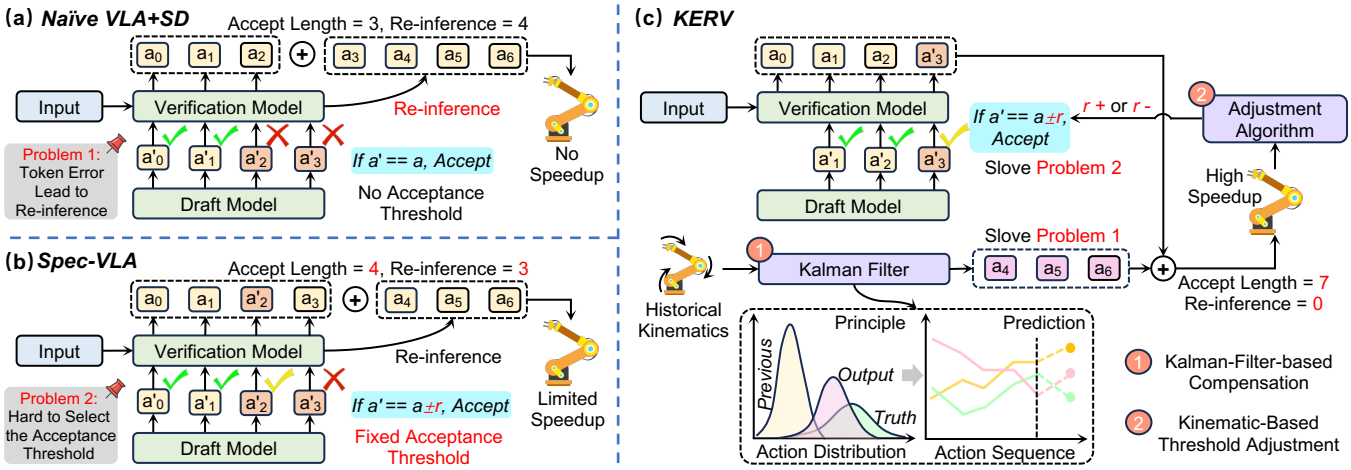

Figure 1: (a) Naive VLA+SD *vs.* (b) SpecVLA *vs.* (c) The Proposed *KERV*

KERV across various specific environments (e.g., LIBERO [15]), analyze its performance drivers, and discuss its optimal configurations. In summary, our contributions are below:

- We reveal the discrepancy between the token-domain (VLA models) and the kinematic-domain (conventional kinematic-based methods) in robot control, demonstrating the feasibility and advantages of their combination.
- Based on these, we take SD as the starting point and combine VLA models and kinematic-based prediction, proposing a kinematic-rectified SD framework termed *KERV*. *KERV* compensates for SD errors via kinematic-based action prediction and leverages kinematics as the metric to achieve adaptive acceptance threshold rectification.
- We implemented *KERV* on existing hardware platforms (tailored to its computational characteristics) and validated the merits of the VLA-kinematics combination via extensive experiments. We believe *KERV* will play a role in future embodied intelligence development and AI-physical combination.

We evaluate *KERV* across diverse experiments and tasks. Experimental results show that compared with naive integration, *KERV* achieves 1.48×~1.57× acceleration, while maintaining the high success rate of VLA models. Additionally, versus SOTA works such as SpecVLA [25], *KERV* achieves 27%~37% speedup with almost no Success Rate loss.

2 Preliminaries

2.1 VLA Models

Model Architecture. Vision-Language-Action (VLA) models comprise three core components [9, 37]: ViT encoders (converting visual data into tokenized representations), an LLM backbone (fusing multi-modal information and enabling reasoning), and an action de-tokenizer (decoding outputs into concrete actions).

Action Decoding Paradigms. Each inference of VLA models produces one action slice A_i, requiring a sequence of action slices to complete a task [35, 36]. Each slice contains 7 DoF: position X, Y, Z of the end gripper, joint rotation angles $\theta_X, \theta_Y, \theta_Z$, and binary gripper control signal G, necessitating 7 autoregressive inference steps per slice. Each DoF is encoded as a token a_j. VLA models predict the most probable token a_j based on the previously tokens $a_{0:j-1}$, visual observations $\mathbb{O}$, language prompts $\mathbb{P}$, and the learnable model parameters $\mathbb{W}$, as Eq. (1) shown:

$$a_j = \arg\max_{a_j} \left[P(a_j \mid a_{0:j-1}, \mathbb{O}, \mathbb{P}, \mathbb{W}) \right], (0 \le j \le 6). \tag{1}$$

Acceleration for VLA Models. VLA acceleration works fall into three categories: model architecture innovation (such as RoboMamba [16], FastV [19], TinyVLA [26] and Edge-VLA [3]), model compression (such as MBQ [11] and QAIL [18]), and runtime optimization (such as VLA-Cache [28], SpecPrune-VLA [24], Efficient-VLA [29], Fastdrive-VLA [5], DeeR-VLA [30], CEED-VLA [21] and MoLe-VLA [34]). However, despite favorable acceleration, existing work still leaves limitations in optimizing the VLA decoding paradigm, requiring decoding optimizations such as SD.

2.2 Speculative Decoding

SD for LLMs. SD comprises a draft model M_{D} and a verification model M_{V}. In the draft phase, M_{D} predicts multiple tokens using hidden states $f_{1:t}$ and embeddings $e_{0:t}$, as shown in Eq. (2). For simplicity, we use $a_{t+1:j-1}$ to denote both embeddings and hidden features. In the verification phase, M_{V} ensures draft token quality; in greedy search, draft token $\hat{a_j}$ is accepted only if strictly matching M_{V}'s predicted token a_j, as shown in Eq. (3):

$$\text{Draft: } \hat{a}_j = M_{\mathrm{D}}(f_{1:t}, e_{0:t}, \hat{a}_{t+1:j-1}). \tag{2}$$

$$\text{Verify: } a_j = M_{\mathrm{V}}(a_0 \sim \hat{a}_{j-1}, \mathbb{P}, \mathbb{W}), \begin{cases} \text{If } a_j = \hat{a}_j, \text{Accept.} \\ \text{If } a_j \neq \hat{a}_j, \text{Resample.} \end{cases} \tag{3}$$

SD has evolved in three stages: pioneering works (e.g., Medusa [4] and Medusa-CTC [27]) introduced multi-head parallel generation with tree-attention verification; evolutionary frameworks (e.g., EAGLE series [12, 13]) enhanced draft modeling for better quality and speedups; recent studies (e.g., EAGLE-3 [14] and HASS [33]) integrated training-time testing to further boost capabilities.

SD for VLA Models. However, integrating SD with VLA models is non-trivial. Naive integration (shown in Fig. 1(a)) faces two key challenges: (1) SD relies on computationally expensive re-inference to address token errors, which is high-cost. (2) To mitigate token errors, the acceptance threshold in SD requires careful adjustment. While Spec-VLA [25] explores the determination of acceptance

Table 1: Case Studies on Naive Integration of VLA and SD

Env.	AR				Naive VLA+SD				
	SR	Speed	Step	T(s)	SR	Speed	Step	*T*(s)	AFEP
Goal	77.0%	1×	157.6	0.188	76.2%	0.86×	159.2	0.217	2.04
Object	71.2%	1×	191.7	0.197	68.6%	0.96×	195.9	0.200	1.75
Spatial	82.8%	1×	126.9	0.198	82.8%	0.98×	127.3	0.201	1.59
Long	54.4%	1×	393.2	0.190	50.2%	0.91×	400.7	0.217	1.67

threshold (shown in Fig. 1(b)), it still relies on re-inference to mitigate errors, which limits the speedup. Furthermore, it adopts a fixed acceptance threshold, overlooking the complexity and dynamics of various embodied intelligence environments.

2.3 Kinematic-Based Robot Control

Principles of Kinematic-Based Methods. Traditional robotic control [1, 8] takes kinematic information (e.g., joint angles, link displacements, motion velocities) as input to directly regulate robot movements, with no token representation involved. It centers on pre-defined mathematical models and control laws, which require human design. Moreover, it relies on accurate robot dynamic modeling and real-time sensor feedback to achieve precise trajectory tracking and motion execution. Unlike VLA models, it lacks the capability for long-term task planning or environmental perception.
Kalman-Filter-Based Robot Control. The Kalman Filter (KF) is a classic technique in traditional control methods [2, 6]. It updates control signals based on the gaps between the prior action distribution and the true value. In robotic control, it leverages kinematic constraints to predict future action states, achieving high accuracy in most scenarios. This lightweight, real-time optimization aligns with the efficiency requirements of kinematic-based control, solidifying its role as a staple for precise action regulation.

3 Token-Kinematic Discrepancy

In this section, we analyze SD-induced errors and the associated acceptance threshold, identify the discrepancy between the token-domain and kinematic-domain paradigms, and demonstrate the insights of combining VLA models with kinematic-based methods.

3.1 Discrepancy about Errors

Motivation ①: SD relies on high-cost re-inference to address generation errors, hindering inference speed. In contrast, kinematic methods enable fast generation with small, speed-independent errors.
Token-Domain Analysis about Generation Errors. We naively adapted SD via the EAGLE framework [13] and OpenVLA [9] model,

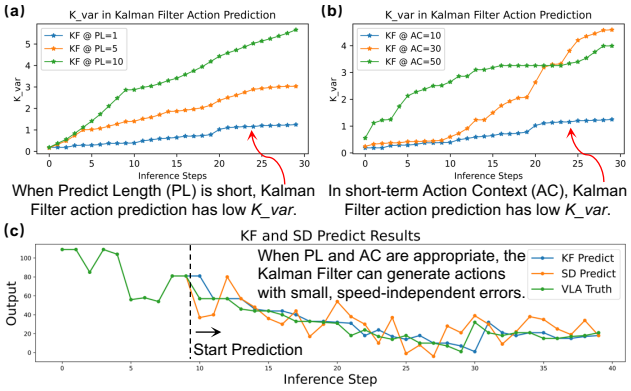

Figure 2: Discrepancy about Errors

evaluating Success Rate (SR), speed, average inference steps, and per-step latency (T). We further report SD's Average First Error Position (AFEP). The results in Tab. 1 show that prediction errors (AFEP 1.59~2.04) force draft token abandonment and subsequent re-inference, leading to lower speed than AR (0.188s~0.198s v.s. 0.200s~0.217s). Clearly, acceleration hinges on properly addressing SD-generated errors with a lower-cost compensation approach instead of re-inference.
Kinematic-Domain Analysis about Prediction Errors. Fortunately, Kalman Filter (KF) features ultra-low computational overhead and fast action prediction, making them ideal alternatives to re-inference. While KF also incurs certain errors, so we propose K_{var} to characterize the kinematic variability, as shown in Eq. (4):

$$K_{\text{var}} = \sum_{step} \left\| action_j^{\text{correct}} - action_j^{\text{error}} \right\|_1, \ (0 \le j \le 6), \quad (4)$$

The Action Context (AC) and Predict Length (PL) will affect KF's prediction errors. We first test KF under various Prediction Length (PL) and the results are shown in Fig. 2 (a). When KF only predicting 1 step ahead (@ PL=1), it achieve minimal K_{var}. When PL increases, the actions predicted by KF become imprecise. Additionally, we test how action context (AC) impacts KF performance. As Fig. 2 (b) shows, KF's K_{var} is minimized in short-term AC (@ AC=10). Beyond that, the K_{var} increases as the AC grows. As Fig. 2(c) shows, with appropriate PL and AC, KF outperforms SD's draft model in action accuracy, and its errors do not impact speed.
Insight ①: KF can be employed to predict actions for compensating SD errors, thereby avoiding computationally expensive re-inference and enabling end-to-end acceleration.

3.2 Discrepancy about Acceptance Threshold

Motivation ②: Existing work adopts a fixed relaxed acceptance threshold for SD errors. In contrast, from a kinematic perspective, some accepted errors should be discarded, and the acceptance threshold should be dynamically adjusted.
Token-Domain Analysis about Acceptance Threshold. Existing work like Spec-VLA [25] adopts a fixed relaxed acceptance threshold, forcing the VLA model to accept draft tokens with errors less than r (r means the relaxed threshold). However, determining r for diverse embodied tasks and environments remains challenging. Experiments show that a large r will increase inference steps and even cause task failure, and a small r will lead to limited speedup. Furthermore, even for a specific task, a fixed threshold is irrational:

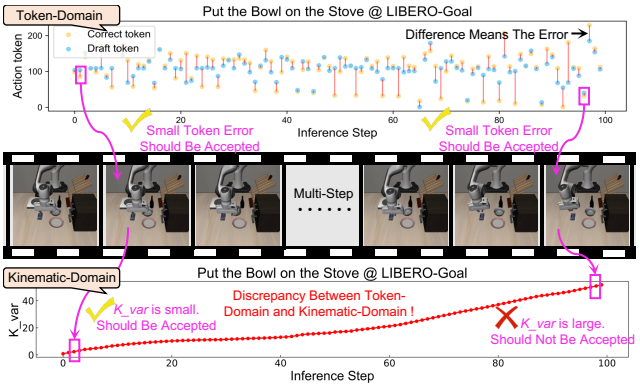

Figure 3: Discrepancy about Acceptance Threshold

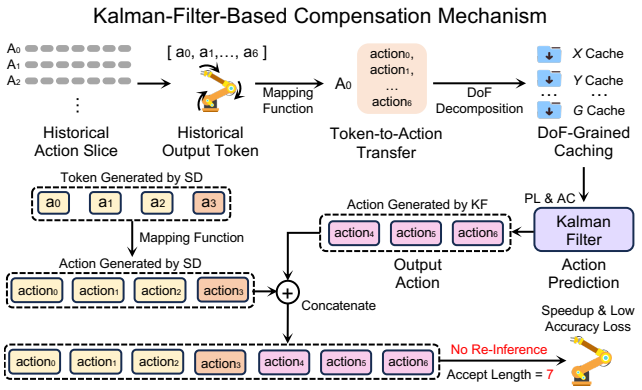

Figure 4: KF-Based Compensation Mechanism in *KERV*

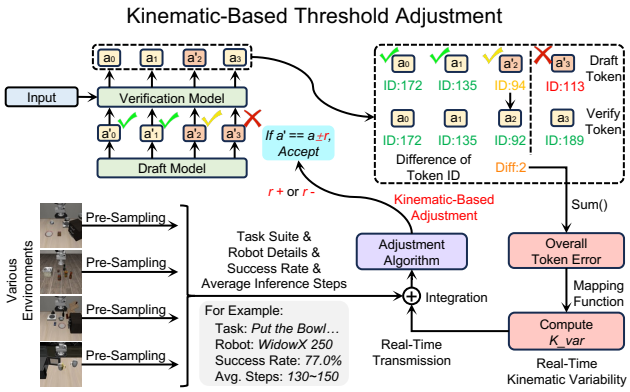

Figure 5: Kinematic-Based Threshold Adjustment in *KERV*

as errors accumulate, the more inference steps there are, the more erroneous tokens that should no longer be accepted.

Kinematic-Domain Analysis about Acceptance Threshold. From a kinematic perspective, we analyze the relaxed acceptance threshold. Specifically, we record the SD errors and corresponding K_{var} during inference (as shown in Fig. 3). We select two representative time steps. At the first case step, the SD-generated token error is small (error=2) and would be accepted under Spec-VLA's relaxed acceptance threshold (r=9). At this point, K_{var} is small, indicating this error is acceptable. At the second case step, the SD-generated token error remains small (error=1) and would also be accepted under this fixed threshold. However, the corresponding K_{var} is large, requiring rejection of this erroneous token, revealing a token-kinematic discrepancy in the acceptance threshold. This indicates that fixed thresholds perform poorly, showing the inadequacy of determining such thresholds solely in the token domain.

Insight ②: The acceptance threshold determined solely from the token domain is suboptimal and demands dynamic adjustment, while kinematic variability serves as a suitable metric to guide this process.

4 *KERV* Framework

The aforementioned analysis and insights drive us to combine VLA models and conventional kinematic-based methods. In this section, we will detail how we combine them and leverage them to address the two key challenges of SD within the *KERV* framework.

4.1 KF-Based Compensation Mechanism

From Insight ①, to address the challenge of re-inference when facing token errors in SD, we combine kinematic-based methods and VLA models, proposing a compensation mechanism. As Fig. 4 illustrates, the core idea is that when an SD error occurs, we use KF to complete action prediction for the remainder of the current action slice instead of re-inference, thereby achieving acceleration.

We first build a mapping function $Map(\cdot,\cdot)$ of the VLA model's token-action transfer, as shown in Eq. (5). $Map(\cdot)$ takes the output token A_i and the pre-sampled action distribution $D_{action}^{\text{Norm-key}}$ of the robotic arm as input, and outputs the corresponding action signals. Norm-key is an inherent statistic of the robotic arm.

$$action_j = Map\big((A_i \mid \mathbb{O},\mathbb{P},\mathbb{W});\ D_{action}^{\text{Norm-key}}\big),\ (0 \le j \le 6). \quad (5)$$

Then, we build caching for these actions in DoF granularity, represented as $\text{Cache}_X, \text{Cache}_Y, \cdots, \text{Cache}_G$. We use these caches as the historical input sequence required by KF to enable accurate action prediction, as shown in Eq. (6). $\mathcal{I}, \mathcal{M}, \mathcal{P}$ is the initial parameters of KF. p means the first error position of SD. We set the predicted length (PL) to 1 and the action context (AC) to 10.

$$Kalman^{PL}\big(\text{Cache}_{X\sim G}^{AC};(\mathcal{I},\mathcal{M},\mathcal{P})\big) = \begin{cases} action_{0\sim p}^{\text{KF}}, \text{Discard} \\ action_{p\sim 6}^{\text{KF}}, \text{Keep} \end{cases} \quad (6)$$

After that, we concatenate the actions from SD ($action_{0\sim p}^{\text{SD}}$) and the actions generated by KF ($action_{p\sim 6}^{\text{KF}}$) to form a complete action slice, and send it to the robotic arm. In the next step, the VLA model re-encodes environmental information and re-prefills for the next generation. Thus, the proposed compensation mechanism does not require modifying the KV Cache. To maintain a short PL for KF's prediction accuracy, we activate the compensation mechanism intermittently. Therefore, after each time of compensation, we use SD for the next n steps with KF disabled. We set $n = 4$ and discuss it in Section 6.

4.2 Kinematic-Based Threshold Adjustment

From Insight ②, to address the challenge of determining the acceptance threshold, we propose a kinematic-based threshold adjustment strategy. As shown in Fig. 5, we compute the ID difference between true tokens and those accepted via relaxed thresholds. Note that unaccepted error tokens do not participate in the calculation. Then, we aggregate the differences at each step and convert them to kinematic variability through the mapping function to calculate K_{var}. Moreover, we pre-sample in diverse environments to gather the necessary information, including the task suite, robot details, success rate, and average inference steps.

After that, we use this information to determine the appropriate maximal and minimal thresholds $r_{\max}$ and $r_{\min}$, as well as the appropriate algorithm parameters τ and φ, and build them into look-up tables (for most tasks, we set $r_{\max}$=15 and $r_{\min}$=5). We integrate these lookup tables and kinematic variability, then feed them into an adjustment algorithm that takes them as inputs and outputs acceptance threshold adjustment guidance. The details of the adjustment algorithm are shown in Alg. 1. This kinematic-based strategy can automatically adjust the acceptance threshold at runtime, enabling better preservation of the VLA model's high success rate during the SD process.

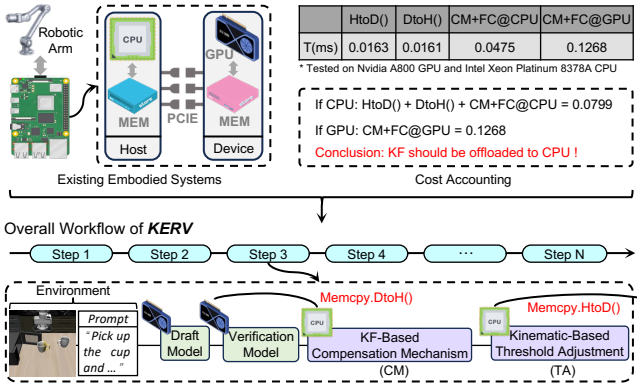

Figure 6: System Implementation of *KERV*

5 System Implementation

This section explores the efficient implementation of *KERV* on existing hardware platforms from a computational perspective based on CPU-GPU collaboration.

5.1 GPU-Side Draft and Verify

Both the draft model and verification model involve substantial computation. Measurements show the draft model requires 0.07 GFLOPs per inference, while the verification model consumes 3.92 TFLOPs per inference. We therefore adopt GPU acceleration for their inference processes. Owing to the draft model's smaller size, its GPU memory footprint is significantly lower than that of the verification model. The draft model incurs a memory overhead of 700MB, while the 7B verification model consumes 15GB of memory, leading to a negligible footprint for the former.

5.2 CPU-Side Compensation and Adjustment

In *KERV*, the KF-Based compensation mechanism and the kinematic-based threshold adjustment involve extensive logical decisions but small FLOPs, which suggests they may be more efficient on the CPU than on the GPU. However, CPU execution requires multiple CPU-GPU data transfers, introducing extra overhead, which prompted detailed benchmarking. As shown in Fig. 6, the existing embodied system integrates both CPU and GPU. On an Nvidia A800 GPU and an Intel Xeon Platinum 8378A CPU, we measured Host-to-Device (HtoD) and Device-to-Host (DtoH) memory copy latencies, implemented two parts, and quantified their latencies. Even with additional memory copy overhead, CPU-executed latency is still lower than that of the GPU. Thus, for speed optimization, we offload these two components to the CPU.

Algorithm 1 Adjustment Algorithm

Require: Kinematic Variability K_{var}, Acceptance Threshold Maximum and Minimum $r_{\max}, r_{\min}$, Parameters Look-up Table Generated by Pre-Sampling $[\tau_1, \tau_2, \cdots, \tau_n]$, $[\varphi_1, \varphi_2, \cdots, \varphi_n]$.
Ensure: Task T, Robot $Robot$, Success Rate R and Average Inference Steps S
Searching τ and φ in $[\tau_1, \cdots, \tau_n]$ and $[\varphi_1, \cdots, \varphi_n]$ based on $[T, Robot, R, S]$
if $\tau \in [\tau_1, \tau_2, \cdots, \tau_n]$ and $\varphi \in [\varphi_1, \varphi_2, \cdots, \varphi_n]$ **then**
  for Each Step t **do** Compute $\Delta K_{\text{var}}^t = K_{\text{var}}^t - K_{\text{var}}^{t-1}$
    if $\Delta K_{\text{var}} \neq 0$ **then**
      $\Delta r^t = (r_{\max} - r_{\min}) * exp((-\frac{\Delta K_{\text{var}}^t}{K_{\text{var}}^S})^{\varphi})$
    else continue
    end if
    Update Acceptance Threshold: $r^{t+1} \leftarrow r^t + \Delta r^t$
    if $r^{t+1} \leq r_{\min}$ **then break**
    end if
  end for
end if

5.3 Overall Workflow of *KERV*

KERV is implemented in ~5000 lines of code. Fig. 6 outlines its workflow: the environmental visual and textual prompt is encoded as input to the draft model, which autoregressively outputs tokens. The verification model evaluates these tokens with a relaxed acceptance threshold. The verification model evaluates these tokens using a relaxed acceptance threshold. The results are then transferred to the CPU via DtoH memory copy for KF-based compensation and kinematic-based threshold adjustment, before being sent back to the GPU via HtoD memory copy for the next step.

6 Experiments

6.1 Setup

Following OpenVLA [9], we tested *KERV* framework on the LIBERO benchmark [15]. We utilize four task suites, including LIBERO-Object, LIBERO-Spatial, LIBERO-Goal, and LIBERO-Long. Each suite contains 10 tasks. For each task, we conduct 50 trials for testing. We utilize the finetuned OpenVLA as the verification models and build a single LLaMA block [23] as a draft model. We trained the draft models based on the DeepSpeed [20] framework. The training process was completed in 12 hours using 2× NVIDIA A800 (40G) GPUs. We inherit the tree decoding mechanism from Eagle-2 [12]. For tree-decoding, we set the maximum nodes to 50, the tree depth to 4, and used the top 8 tokens to construct the draft tree. Given that Spec-VLA [25] is the only existing work integrating VLA and SD, we adopt it alongside naive VLA+SD as baselines. We select an Nvidia A800 GPU and an Intel Xeon Platinum 8378A CPU as the testing hardware platform.

6.2 Evaluation Results

End-to-End Results. We report the Success Rate (SR), Speedup, Average First Error Position (AFEP), and average inference steps, as Tab. 2. The results show that compared with naive combination, *KERV* achieves 1.48×~1.57× speedup, without SR loss. Compared with Spec-VLA under various acceptance thresholds (r=9/15/20), *KERV* achieves 27%~37% acceleration, with only 1.5% and 0.4% SR decrease on two environments. Moreover, in all four environments, *KERV* has the smallest average inference steps. All these results demonstrate that combining kinematics with VLA models enables a better speed-accuracy Pareto Frontier.

Example Presentation. We present an example to illustrate KERV's details in the SD process. As shown in Fig. 7, the draft model generates tokens [140, 149, 183] for the first case step: yellow number 149 denotes an erroneous token accepted by the relaxed threshold, green number 140 a fully correct token, and red number 183 an erroneous token rejected for exceeding the threshold. After verification, the verification model corrects 183 to 128, and these tokens are decoded into actions (action scope -1~1). The Compensation Mechanism (hereinafter referred to as CM) compensates for residual positions using predicted actions (pink numbers), with the final output being the concatenation of SD-generated actions and CM-predicted actions. At this step, the acceptance threshold r is set to 14 under the Threshold Adjustment (TA) strategy. For the second case step, r is adjusted to 5, demonstrating TA's dynamic effect.

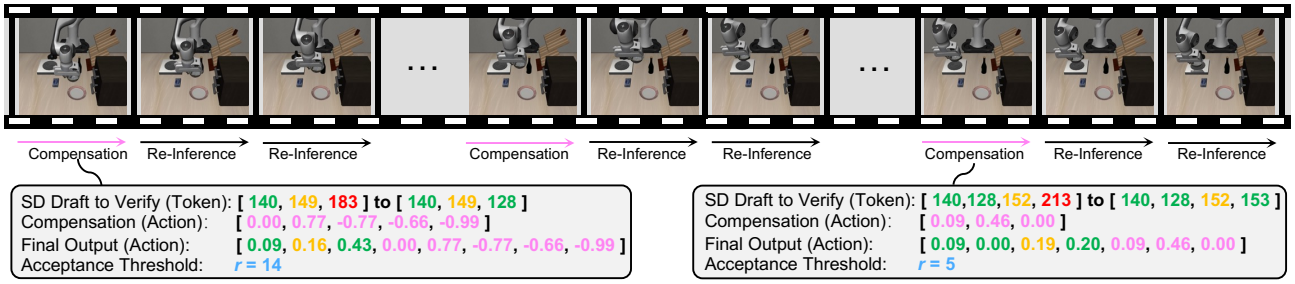

Figure 7: An Experimental Example of *KERV* on LIBERO-Goal Environment

Ablation Studies. We performed ablation experiments on two components of *KERV*: CM and TA, and the results are shown in Tab. 4. The four environments show the same trend. With only the CM, the system achieves high speed. However, as the threshold remains unadjusted, it accepts more erroneous SD-generated tokens, leading to significant accuracy loss. In contrast, with only TA, the system automatically adjusts the threshold based on dynamic changes, thus yielding favorable accuracy. Without CM, however, re-inference is unavoidable, resulting in limited acceleration.

6.3 Discussion

Hyperparameters. The hyperparameters of *KERV* include the prediction length (PL), the action context (AC) of KF, and the number of SD steps (n) executed after each time of KF. We evaluate *KERV*'s performance across different hyperparameters on LIBERO-Goal, with the results presented in Fig. 8. From Fig. 8 (a), when n<4, the SR decreases rapidly. When n>4, the acceleration ratio decreases gradually due to the increase in SD steps, and the difference in accuracy is not large. Therefore, n=4 is the optimal choice. From Fig. 8 (b), as AC varies, the speed fluctuates slightly, while SR decreases with increasing AC. Thus, we use AC=10 as the basis selection for *KERV*.

Table 2: End-to-End Results of *KERV* and Baselines

Env.	Method	SR	Speed	AFEP	Steps	Hardware
Goal	Naive VLA+SD	76.2%	1.00×	2.04	159.2	GPU
	SpecVLA (r=9)	75.4%	1.19×	2.97	157.3	GPU
	SpecVLA (r=15)	71.0%	1.23×	3.63	166.8	GPU
	SpecVLA (r=20)	64.2%	1.21×	4.23	176.3	GPU
	KERV	**75.6%**	**1.54×**	**4.73**	**153.5**	CPU+GPU
Object	Naive VLA+SD	68.6%	1.00×	1.75	195.9	GPU
	SpecVLA (r=9)	70.0%	1.09×	3.25	200.0	GPU
	SpecVLA (r=15)	62.4%	1.10×	3.91	214.5	GPU
	SpecVLA (r=20)	58.0%	1.10×	4.18	221.5	GPU
	KERV	**72.3%**	**1.49×**	**4.71**	**186.8**	CPU+GPU
Spatial	Naive VLA+SD	82.8%	1.00×	1.59	127.3	GPU
	SpecVLA (r=9)	**85.2%**	1.25×	3.27	124.9	GPU
	SpecVLA (r=15)	80.4%	1.26×	3.80	128.7	GPU
	SpecVLA (r=20)	77.8%	1.24×	4.14	133.3	GPU
	KERV	83.7%	**1.57×**	**4.67**	**120.9**	CPU+GPU
Long	Naive VLA+SD	50.2%	1.00×	1.67	400.7	GPU
	SpecVLA (r=9)	**49.2%**	1.12×	2.82	408.9	GPU
	SpecVLA (r=15)	36.2%	1.13×	3.63	439.6	GPU
	SpecVLA (r=20)	25.8%	1.10×	4.13	464.6	GPU
	KERV	48.8%	**1.48×**	**4.64**	**391.2**	CPU+GPU

Table 3: Ablation Studies of *KERV*

KERV		Goal		Object		Spatial		Long	
CM	TA	SR	Speed	SR	Speed	SR	Speed	SR	Speed
✔	✗	72.6%	1.57×	68.1%	1.48×	80.9%	1.59×	46.9%	1.48×
✗	✔	77.0%	1.17×	72.9%	1.07×	85.0%	1.21×	49.6%	1.11×
✔	✔	75.6%	1.54×	72.3%	1.49×	83.7%	1.57×	48.8%	1.48×

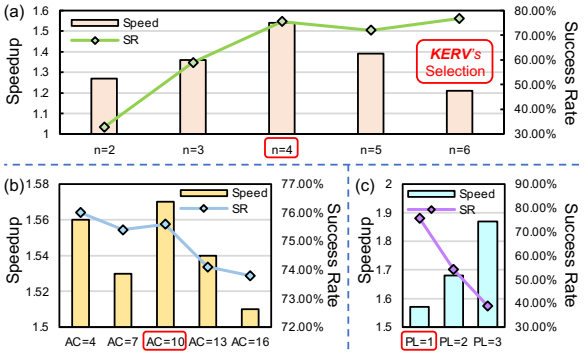

Figure 8: Experiments of Hyper-Parameters in *KERV*

From Fig. 8 (c), as the PL increases, SR dropped sharply. This is because KF can only maintain accurate predictions in a short context, and when the context expands, it is equivalent to making the next prediction with the predicted value with error, resulting in a sharp decrease in SR. Therefore, we select PL=1 in *KERV*.

Hardware Implementation Analysis. We test the end-to-end time overhead on different hardware platforms in four environments. The Results are shown in Tab. 4. Compared with GPU-only, our proposed CPU+GPU implementation achieves lower end-to-end latency, showing 7%~13% acceleration contribution.

Generability. Due to the limited generalization of the VLA model itself, it is difficult to adapt to a variety of real-world robotic arms. Therefore, we do not do real machine adaptation, but pay more attention to how to combine the VLA model with kinetic methods.

7 Conclusion

In this paper, we first analyze the discrepancy between token-domain and kinematic-domain to derive insights. Then we target SD optimization and propose *KERV* framework, which contains a Kalman-Filter-based compensation mechanism and a kinematic-based threshold adjustment strategy. Experimental results show that *KERV* achieves 27%~37% acceleration with nearly no SR loss.

8 Acknowledgment

This paper is supported by National Natural Science Foundation of China (NSFC) Projects (No. 62227809).

Table 4: Hardware Implementation Analysis of *KERV*

Detials	Goal	Object	Spatial	Long
KERV on GPU @ 50 Trials	12473.4 s	14073.6 s	9111.8 s	31308.7 s
KERV on CPU+GPU @ 50 Trials	11206.6 s	13147.9 s	8183.2 s	27607.1 s
Acceleration of CPU+GPU	1.11×	1.07×	1.11×	1.13×

References

[1] Andrea Bajo and Nabil Simaan. 2011. Kinematics-based detection and localization of contacts along multisegment continuum robots. IEEE Transactions on Robotics 28, 2 (2011), 291–302.

[2] Pieter M Blok, Koen van Boheemen, Frits K van Evert, Joris IJsselmuiden, and Gook-Hwan Kim. 2019. Robot navigation in orchards with localization based on Particle filter and Kalman filter. Computers and electronics in agriculture 157 (2019), 261–269.

[3] Paweł Budzianowski, Wesley Maa, Matthew Freed, Jingxiang Mo, Winston Hsiao, Aaron Xie, Tomasz Młoduchowski, Viraj Tipnis, and Benjamin Bolte. 2025. Edgevla: Efficient vision-language-action models. arXiv preprint arXiv:2507.14049 (2025).

[4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. arXiv preprint arXiv:2401.10774 (2024).

[5] Jiajun Cao, Qizhe Zhang, Peidong Jia, Xuhui Zhao, Bo Lan, Xiaoan Zhang, Xiaobao Wei, Sixiang Chen, Zhuo Li, Yang Wang, et al. 2025. Fastdrivevla: Efficient end-to-end driving via plug-and-play reconstruction-based token pruning. arXiv preprint arXiv:2507.23318 (2025).

[6] Shen-Yong Chen. 2011. Kalman filter for robot vision: a survey. IEEE Transactions on industrial electronics 59, 11 (2011), 4409–4420.

[7] Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, et al. 2025. Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation. arXiv preprint arXiv:2505.03912 (2025).

[8] Chrysostomos Karakasis and Panagiotis Artemiadis. 2021. F-VESPA: A kinematic-based algorithm for real-time heel-strike detection during walking. In 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 5098–5103.

[9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. Openvla: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246 (2024).

[10] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning. PMLR, 19274–19286.

[11] Shiyao Li, Yingchun Hu, Xuefei Ning, Xihui Liu, Ke Hong, Xiaotao Jia, Xiuhong Li, Yaqi Yan, Pei Ran, Guohao Dai, et al. 2025. Mbq: Modality-balanced quantization for large vision-language models. In Proceedings of the Computer Vision and Pattern Recognition Conference. 4167–4177.

[12] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle-2: Faster inference of language models with dynamic draft trees. arXiv preprint arXiv:2406.16858 (2024).

[13] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. arXiv preprint arXiv:2401.15077 (2024).

[14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. arXiv preprint arXiv:2503.01840 (2025).

[15] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning. Advances in Neural Information Processing Systems 36 (2023), 44776–44791.

[16] Jiaming Liu, Mengzhen Liu, Zhenyu Wang, Lily Lee, Kaichen Zhou, Pengju An, Senqiao Yang, Renrui Zhang, Yandong Guo, and Shanghang Zhang. 2024. Robomamba: Multimodal state space model for efficient robot reasoning and manipulation. arXiv e-prints (2024), arXiv–2406.

[17] Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. 2024. A survey on vision-language-action models for embodied ai. arXiv preprint arXiv:2405.14093 (2024).

[18] Seongmin Park, Hyungmin Kim, Wonseok Jeon, Juyoung Yang, Byeongwook Jeon, Yoonseon Oh, and Jungwook Choi. 2024. Quantization-aware imitation-learning for resource-efficient robotic control. arXiv preprint arXiv:2412.01034 (2024).

[19] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. 2025. Fast: Efficient action tokenization for vision-language-action models. arXiv preprint arXiv:2501.09747 (2025).

[20] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. 3505–3506.

[21] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Yuxin Huang, Han Zhao, Donglin Wang, and Haoang Li. 2025. CEED-VLA: Consistency Vision-Language-Action Model with Early-Exit Decoding. arXiv preprint arXiv:2506.13725 (2025).

[22] Xudong Tan, Yaoxin Yang, Peng Ye, Jialin Zheng, Bizhe Bai, Xinyi Wang, Jia Hao, and Tao Chen. 2025. Think Twice, Act Once: Token-Aware Compression and Action Reuse for Efficient Inference in Vision-Language-Action Models. CoRR abs/2505.21200 (2025). https://doi.org/10.48550/ARXIV.2505.21200 arXiv:2505.21200

[23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023).

[24] Hanzhen Wang, Jiaming Xu, Jiayi Pan, Yongkang Zhou, and Guohao Dai. 2025. Specprune-vla: Accelerating vision-language-action models via action-aware self-speculative pruning. arXiv preprint arXiv:2509.05614 (2025).

[25] Songsheng Wang, Rucheng Yu, Zhihang Yuan, Chao Yu, Feng Gao, Yu Wang, and Derek F Wong. 2025. Spec-vla: speculative decoding for vision-language-action models with relaxed acceptance. arXiv preprint arXiv:2507.22424 (2025).

[26] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. 2025. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. IEEE Robotics and Automation Letters (2025).

[27] Zhuofan Wen, Shangtong Gui, and Yang Feng. 2024. Speculative decoding with CTC-based draft model for LLM inference acceleration. Advances in Neural Information Processing Systems 37 (2024), 92082–92100.

[28] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. 2025. Vla-cache: Towards efficient vision-language-action model via adaptive token caching in robotic manipulation. arXiv preprint arXiv:2502.02175 (2025).

[29] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. 2025. EfficientVLA: Training-Free Acceleration and Compression for Vision-Language-Action Models. arXiv preprint arXiv:2506.10100 (2025).

[30] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. 2024. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. Advances in Neural Information Processing Systems 37 (2024), 56619–56643.

[31] Jingyi Zhang, Jiaxing Huang, Sheng Jin, and Shijian Lu. 2024. Vision-language models for vision tasks: A survey. IEEE transactions on pattern analysis and machine intelligence 46, 8 (2024), 5625–5644.

[32] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & verify: Lossless large language model acceleration via self-speculative decoding. arXiv preprint arXiv:2309.08168 (2023).

[33] Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. 2024. Learning harmonized representations for speculative sampling. arXiv preprint arXiv:2408.15766 (2024).

[34] Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, Xiaowei Chi, Gaole Dai, Li Du, Yuan Du, and Shanghang Zhang. 2025. Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. arXiv preprint arXiv:2503.20384 (2025).

[35] Shulai Zhang, Ao Xu, Quan Chen, Han Zhao, Weihao Cui, Ningxin Zheng, Haibin Lin, Xin Liu, and Minyi Guo. 2025. Boosting Embodied AI Agents through Perception-Generation Disaggregation and Asynchronous Pipeline Execution. arXiv preprint arXiv:2509.09560 (2025).

[36] Wenxin Zheng, Boyang Li, Bin Xu, Erhu Feng, Jinyu Gu, and Haibo Chen. 2025. Leveraging OS-Level Primitives for Robotic Action Management. arXiv preprint arXiv:2508.10259 (2025).

[37] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Conference on Robot Learning. PMLR, 2165–2183.